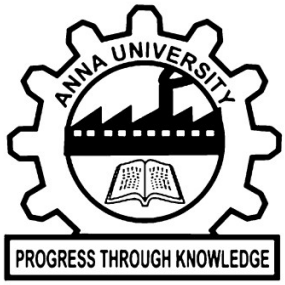
STUDENT MANAGEMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

DEEPAN RAJA G - 810023205049



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,
ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,
ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that Summer internship project report **STUDENT MANAGEMENT SYSTEM** is the bonafide work of **DEEPAN RAJA G (810023205049)** has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.Tech INFORMATION TECHNOLOGY** during the academic year 2026-2027.

FACULTY INCHARGE **HEAD OF THE DEPARTMENT**

ABSTRACT

This report presents the work carried out during a one-month industrial internship at Fantasy Solution, Tiruchirappalli, from 01 June 2026 to 30 June 2026. The internship combined a structured training programme in Java and enterprise application development with the design and development of a Student Management System.

The training programme covered core and advanced Java, object-oriented programming, collections, multithreading, JDBC, Servlets and JSP, the Spring and Spring Boot frameworks, Hibernate, Spring Security, JUnit testing, Git version control and deployment on Apache Tomcat.

The Student Management System automates the day-to-day operations of an academic institution: maintenance of student, faculty and course records, course enrolment, attendance tracking, marks entry, grade and result processing, and reporting. It follows a layered architecture with a relational MySQL database, role-based access for administrators, faculty and students, and transactional processing that keeps enrolment records accurate. The report explains the problem, requirements, design with use case and entity-relationship diagrams, implementation, testing strategy and results.

Beyond technical skills, the internship provided exposure to professional practices such as teamwork, documentation, code review and time management, and has strengthened the foundation for a career in software development.

Keywords: *Student Management System*, *Java*, *Spring Boot*, *Hibernate*, *MySQL*, *Internship*

TABLE OF CONTENTS

1. Introduction..... 1
2. Organisation Profile..... 3
3. Internship Training Programme..... 5
4. Project Overview..... 7
5. System Analysis and Requirements..... 9
6. System Design..... 11
7. Implementation..... 15
8. Testing..... 18
9. Application Walkthrough and Results..... 20
10. Learning Outcomes and Challenges..... 24
11. Conclusion and Future Scope..... 26
References..... 26

LIST OF FIGURES

Figure 3.1: Phases of the internship programme (June 2026) 5
Figure 6.1: Layered architecture of the Student Management System 11
Figure 6.2: Use case diagram 12
Figure 6.3: Entity-relationship diagram 12
Figure 6.4: Flowchart of attendance, marks entry and result processing 14
Figure 9.1: Login page 20
Figure 9.2: Dashboard with summary tiles and quick actions 20
Figure 9.3: Student list with search, filters and status 21
Figure 9.4: Course enrolment screen 21
Figure 9.5: Attendance and marks entry screen 22
Figure 9.6: Reports screen and student result view 22

LIST OF TABLES

Table 1.1: Internship details 1
Table 2.1: Contact details of the organisation 3
Table 2.2: Technology areas handled by the organisation 4
Table 3.1: Technologies studied and their use in the project 6
Table 4.1: Feasibility analysis 8
Table 5.1: Functional requirements 9
Table 5.2: Non-functional requirements 9
Table 5.3: Sample academic rules 10
Table 5.4: Development and deployment environment 10
Table 6.1: Table STUDENT 13
Table 6.2: Table MARKS 13
Table 7.1: Modules of the Student Management System 15
Table 8.1: Levels of testing 18
Table 8.2: Representative test cases and expected results 18
Table 9.1: Coverage of functional requirements by application screens 22
Table 9.2: Project objectives and outcomes 23
Table 10.1: Challenges faced and solutions adopted 24

CHAPTER 01

1. Introduction

1.1 Background

An internship is a structured period of professional work experience in which a student applies classroom knowledge to real problems under the guidance of working professionals. Learning happens through doing: the intern joins a team, follows its practices, takes on tasks with real expectations and grows from the outcome. This complements the theoretical foundation built in the classroom.

This report documents the industrial internship completed at **Fantasy Solution, Tiruchirappalli**, between **01 June 2026 and 30 June 2026**, as part of the B.Tech Information Technology programme. The internship combined a focused training programme in Java and enterprise application development with the design and development of a practical product, the **Student Management System**.

1.2 Objectives of the Internship

- To understand how a professional software organisation plans, builds, tests and delivers applications.
- To strengthen core and advanced Java skills, including object-oriented design, collections, multithreading and database connectivity.
- To gain hands-on experience with enterprise technologies such as Servlets, JSP, Spring Boot, Hibernate and Spring Security.
- To apply these skills to a complete project, from requirement analysis and design to implementation, testing and deployment.
- To develop professional habits: communication, documentation, version control, code review and teamwork.

1.3 Scope of the Work

The internship covered two connected streams of work. The first was a guided training stream in which fundamental and advanced Java concepts were studied and practised through exercises. The second was the project stream, in which these concepts were applied to build a Student Management System that automates student records, course enrolment, attendance tracking, marks and result processing, and reporting for an academic institution. This report concentrates on the knowledge gained, the design decisions taken and the engineering practices followed.

1.4 Internship at a Glance

The key details of the internship, as recorded in the official internship annexure, are summarised below.

Table 1.1: Internship details

Particular	Details
Name of the industry	Fantasy Solution, #16, Samnath Plazza, 3rd Floor, Melapudur, Trichy – 620 001
Industrial expert (Joint Supervisor)	Mr. Bragatheesh
Qualification of the expert	MCA

Particular	Details
Designation of the expert	Software Developer
Title of the project work	Student Management System
Duration of the internship	01-06-2026 to 30-06-2026
Name of the student	Deepan Raja G
Registration number	810023205049
Department	B.Tech – Information Technology

1.5 Organisation of the Report

- Chapter 2 presents the profile of Fantasy Solution, its services, mission and vision.
- Chapter 3 describes the technical training programme and the technologies studied.
- Chapters 4 and 5 introduce the Student Management System and analyse its requirements.
- Chapter 6 explains the system design, including architecture, use cases and database design.
- Chapters 7 and 8 cover implementation and testing.
- Chapter 9 walks through the application and reviews the results achieved.
- Chapters 10 and 11 reflect on learning outcomes and challenges, and present the conclusion and future scope.

CHAPTER 02

2. Organisation Profile

2.1 About Fantasy Solution

Fantasy Solution is an information technology solution and service provider that delivers technology-enabled solutions to businesses, institutions and individuals. The company focuses on software technology and offers industry solutions, product engineering, software products and platforms, and related services.

Its portfolio includes real-time projects, web designing, web hosting, software development and training. Working under the brand commitment of **"Beyond Technology"**, Fantasy Solution aims to apply information technology to support sustainable development of society while being respected by its employees, shareholders, customers and the community.

2.2 Contact Details

Table 2.1: Contact details of the organisation

Item	Details
Organisation	Fantasy Solution
Address	No. 16, Samnath Plazza, Third Floor, Near BG Naidu Sweets, Melapudur, Trichy – 620 001
Mobile	9043095535
Landline	0431-4971630
Website	www.fantasysolution.in

2.3 Services Offered

Fantasy Solution operates across the full software life cycle: it studies, designs, develops, enhances, customises, implements, maintains and supports different aspects of information technology. Its major service areas are:

- Application development and maintenance, including custom software and enterprise applications.
- E-business solutions, content management systems, payment gateway solutions and time-and-attendance software.
- Professional website design, development and web hosting.
- Mobile application development on the Android platform.
- ERP implementation and consulting, testing and performance engineering.
- IT education and training, including project guidance for students and professionals.

2.4 Technology Expertise

Table 2.2: Technology areas handled by the organisation

Area	Technologies and tools
Software development	.NET, PHP and Java
Web development	Web designing and development, content management systems, web hosting
Mobile	Android application development
Data and analytics	Data mining tools, big data development using R and Python
Research and simulation	MATLAB for image processing, Network Simulator (NS2)
Domain solutions	Electronic health records, payment gateways, appointment reminders, medical transcription

2.5 Mission and Vision

The mission of Fantasy Solution rests on five commitments:

1. Providing high-quality software development services, professional consulting and development outsourcing that improve customers' operations.
2. Making access to information easier and more secure for enterprise business.
3. Improving communication and data exchange between businesses.
4. Providing customers with value for money.
5. Providing employees with meaningful work and opportunities for advancement.

> **NOTE**
> **Vision:** To be a leading IT company for consulting services and the deployment of best-of-breed business solutions to top-tier domestic and international customers.

2.6 Working Environment

The internship began with an orientation in which interns were introduced to the team, the working practices of the organisation and the structure of the programme. Throughout the month the atmosphere was collaborative: senior developers were approachable, doubts were resolved quickly and feedback was given openly. Working inside a professional team helped in understanding workplace etiquette, punctuality, shared responsibility and the importance of clear communication.

CHAPTER 03

3. Internship Training Programme

3.1 Structure and Approach

The internship ran for thirty days and was organised into progressive phases. The early phase built a strong base in Java, the middle phase introduced data and web technologies, and the later phases applied everything in a project supported by modern frameworks, security, testing and deployment practices. Each concept was introduced through a short explanation followed by coding exercises, so that understanding was always tested by practice.

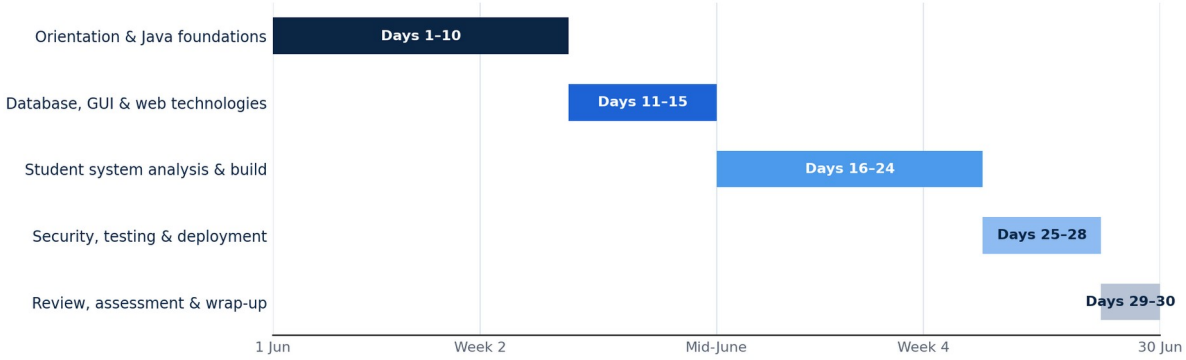


Figure 3.1: *Phases of the internship programme (June 2026)*

3.2 Core Java and Object-Oriented Programming

The programme started with the Java platform, its features and the architecture of the Java Virtual Machine. Data types, variables, operators and control statements were practised through small programs. The emphasis then moved to object-oriented programming: classes and objects, inheritance, polymorphism, encapsulation and abstraction, followed by interfaces, abstract classes and the role of design patterns in building maintainable code.

Exception handling and debugging formed an important part of this phase. Try-catch blocks, custom exceptions and the debugging tools of the Eclipse IDE were used to locate and fix defects systematically instead of by trial and error.

3.3 Advanced Java Concepts

- **Collections Framework:** lists, sets and maps, and choosing the right structure for a problem.
- **Generics:** type-safe, reusable classes and methods.
- **File I/O:** reading from and writing to different kinds of files.
- **Multithreading and concurrency:** thread creation, synchronisation and inter-thread communication.
- **Networking:** socket programming and a simple client-server chat application.

3.4 Database and Web Technologies

Database connectivity was studied with JDBC: connecting Java applications to relational databases, executing SQL queries, handling result sets and implementing CRUD (Create, Read, Update, Delete) operations in a prototype application. JavaFX was used to build simple graphical interfaces. Web

development began with Servlets and JavaServer Pages (JSP), followed by an overview of Java EE capabilities such as web services and the Java Persistence API (JPA).

3.5 Frameworks: Spring and Hibernate

The framework phase introduced the Spring ecosystem. Dependency injection and aspect-oriented programming were practised by configuring a Spring project and its beans. Spring Boot was then used to produce standalone, production-quality applications and a RESTful web service. Hibernate was studied for object-relational mapping and combined with Spring Boot to build a full application with a Hibernate-backed persistence layer. Spring Security introduced authentication and authorisation for protecting application features.

3.6 Quality, Version Control and Deployment

Engineering practices received equal attention. JUnit was used for unit and integration testing, Git for version control and teamwork, and Java web applications were deployed on Apache Tomcat. The programme closed with documentation, project presentations before mentors and peers, and a review of the experience.

3.7 Technology Summary

Table 3.1: Technologies studied and their use in the project

Technology	What was learnt	Role in the Student Management System
Core Java, OOP	Classes, inheritance, interfaces, exceptions	Domain model: Student, Course, Enrollment, Attendance, Marks
Collections, Generics	Lists, sets, maps, type-safe code	In-memory handling of search results and reports
JDBC, SQL	Queries, result sets, CRUD	Foundation for the database layer
Servlets, JSP	Request handling, dynamic pages	Web screens and form handling
Spring, Spring Boot	Dependency injection, REST services	Controllers, services and application configuration
Hibernate, JPA	Object-relational mapping	Mapping entities to MySQL tables
Spring Security	Authentication, authorisation	Role-based access for admin, faculty and student
JUnit	Unit and integration testing	Verification of grading and attendance rules
Git, Tomcat	Version control, deployment	Source management and application hosting

CHAPTER 04

4. Project Overview

4.1 Introduction to the Student Management System

An educational institution handles a continuous flow of academic information: students are admitted, enrolled in courses, marked present or absent, assessed and finally promoted or graded. A Student Management System (SMS) is a software application that automates these activities. It maintains student and faculty records, controls course enrolment, tracks attendance, processes marks into grades and results, and produces reports that help administrators and faculty make decisions.

The project assigned during the internship was to design and develop such a system using the Java technology stack studied in the training programme. It provided an opportunity to follow a complete software development life cycle on a problem that is easy to understand yet rich enough to require careful design.

4.2 Problem Statement

Institutions that depend on paper registers and spreadsheets struggle with slow searching, duplicate or missing entries, inconsistent attendance records, manual and error-prone result computation, and the absence of timely reports. The problem addressed by this project is to provide a secure, easy-to-use and reliable web-based system that replaces manual record keeping with an integrated digital workflow.

4.3 Existing System and its Limitations

- **Manual records:** student details are written in registers and files, which are slow to search and easy to lose.
- **Scattered data:** attendance, marks and personal details are kept separately, so a complete student profile is hard to assemble.
- **Error-prone calculation:** totals, grades and attendance percentages are computed by hand.
- **Delayed information:** students learn about attendance shortage or results late.
- **Limited reporting:** preparing class-wise or course-wise statistics takes hours.
- **Weak access control:** anyone with access to the register can alter entries.

4.4 Proposed System

The proposed Student Management System stores all information in a central relational database and exposes it through a role-based web interface. Administrators, faculty and students each see only the functions relevant to them. Enrolment, attendance and marks update the records immediately, percentages and grades are computed by the system, and reports are generated on demand.

4.5 Objectives of the Project

- Maintain a complete, searchable record of students, faculty, departments and courses.
- Automate course enrolment with checks for duplicates and seat capacity.
- Record attendance session by session and calculate attendance percentage automatically.
- Capture assessment marks and compute totals, grades and pass or fail status using configurable rules.

- Provide secure, role-based access for administrators, faculty and students.
- Generate reports on attendance, results and enrolment.

4.6 Scope and Limitations

The system covers student and faculty management, course administration, enrolment, attendance, marks and result processing, and reporting for a single institution. Items outside the present scope, such as fee collection, online examinations and multi-campus synchronisation, are listed as future enhancements in Chapter 11.

4.7 Feasibility Study

Table 4.1: Feasibility analysis

Type	Assessment
Technical	The project uses mature, widely supported open technologies (Java, Spring Boot, Hibernate, MySQL) for which skills were acquired during the training programme.
Economic	All software components are open source and the application runs on modest hardware, so development and running costs are low compared with the manual effort saved.
Operational	The interface follows familiar form-and-table patterns, so staff and students need very little training to use it.
Schedule	The scope was planned to fit the four-week internship by building the system module by module.

CHAPTER 05

5. System Analysis and Requirements

5.1 Development Methodology

An iterative approach was followed. The requirements were first understood in discussion with the industry guide, then the system was designed at a high level and implemented in small modules. Each module was tested and reviewed before the next one was started, which kept the risk of large rework low and allowed feedback from the mentor to be absorbed early.

5.2 Functional Requirements

Table 5.1: Functional requirements

ID	Requirement	Role
FR-01	Users shall log in with a username and password and log out securely.	All
FR-02	The system shall add, update, delete and view student records.	Admin
FR-03	The system shall manage departments, courses and subjects.	Admin
FR-04	The system shall manage faculty records and assign courses to faculty.	Admin
FR-05	The system shall enrol students in courses, rejecting duplicates and full courses.	Admin
FR-06	Faculty shall mark attendance for each class session.	Faculty
FR-07	Faculty shall enter marks for each assessment.	Faculty
FR-08	The system shall compute totals, grades, attendance percentage and result status.	System
FR-09	Students shall view their profile, attendance and results.	Student
FR-10	The system shall generate attendance, result and enrolment reports.	Admin, Faculty

5.3 Non-Functional Requirements

Table 5.2: Non-functional requirements

Attribute	Requirement
Security	Passwords are stored as salted hashes; every page and action is protected by role-based authorisation.
Usability	Clean, consistent screens with clear validation messages and minimal clicks for common tasks.
Reliability	Enrolment and mark updates are transactional, so records never become inconsistent.
Performance	Searches and listings respond quickly through indexed database columns and paging.
Maintainability	Layered architecture with separate controller, service and repository code.

Attribute	Requirement
Portability	Runs on any platform that supports Java and MySQL, with deployment on Apache Tomcat.

5.4 Business Rules

The academic rules are kept as configuration values so that the institution can change them without modifying code. The values below were used as the sample configuration during development.

Table 5.3: Sample academic rules

Rule	Sample value
Minimum attendance	75% of conducted sessions
Pass mark	40% of the maximum score
Grade bands	O: 90 and above · A+: 80–89 · A: 70–79 · B+: 60–69 · B: 50–59 · C: 40–49 · RA: below 40
Mark validity	Marks must lie between 0 and the maximum score of the assessment
Enrolment	A student cannot enrol twice in the same course, and a course cannot exceed its capacity
Registration number	Must be unique for every student

5.5 Hardware and Software Requirements

Table 5.4: Development and deployment environment

Component	Specification
Processor	Intel Core i3 or higher (or equivalent)
Memory	4 GB RAM minimum, 8 GB recommended
Storage	10 GB of free disk space
Operating system	Windows 10/11 or Linux
Language and runtime	Java Development Kit (JDK) 17 or later
Frameworks	Spring Boot, Spring MVC, Spring Security, Hibernate / JPA
Database	MySQL 8
Web server	Apache Tomcat
IDE and tools	Eclipse IDE, Git, Maven, JUnit
Browser	Any modern browser such as Chrome, Edge or Firefox

CHAPTER 06

6. System Design

6.1 Design Approach

The design separates the application into layers so that each part has a single responsibility. The user interface only presents data, controllers receive and validate requests, the service layer enforces business rules, and the data access layer talks to the database. This separation makes the system easier to understand, test and extend, and it directly reflects the layered architecture studied during the Spring and Hibernate training.

6.2 System Architecture

The Student Management System follows a layered, three-tier architecture. A security layer sits beside the controllers and checks the identity and role of every request before it reaches the business logic.

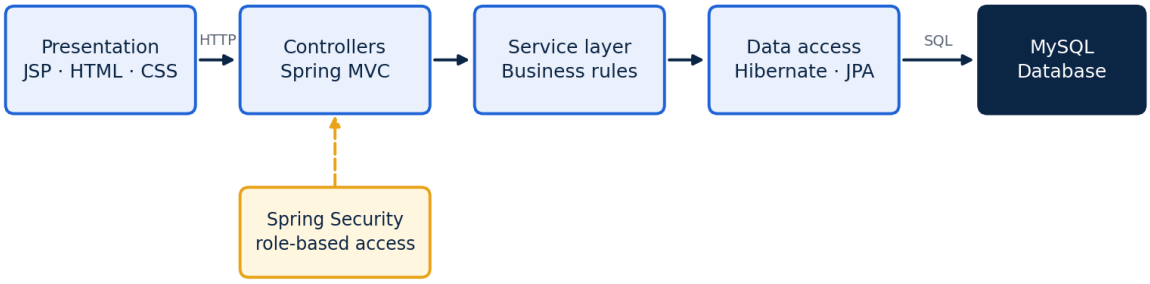


Figure 6.1: *Layered architecture of the Student Management System*

6.3 Use Case Diagram

The use case diagram shows how the three types of users interact with the system. All users must log in; beyond that, each role has its own set of permitted functions.

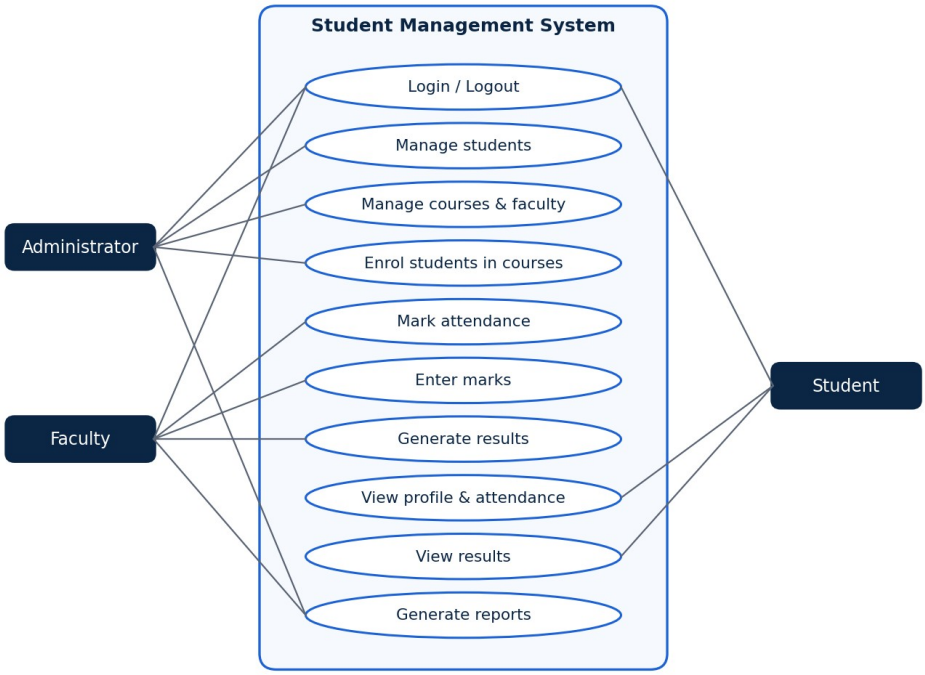


Figure 6.2: *Use case diagram*

6.4 Entity-Relationship Diagram

The database is designed around eight entities. A department offers many courses and has many students; a student is linked to courses through enrolment records; and attendance and marks are recorded against an enrolment.

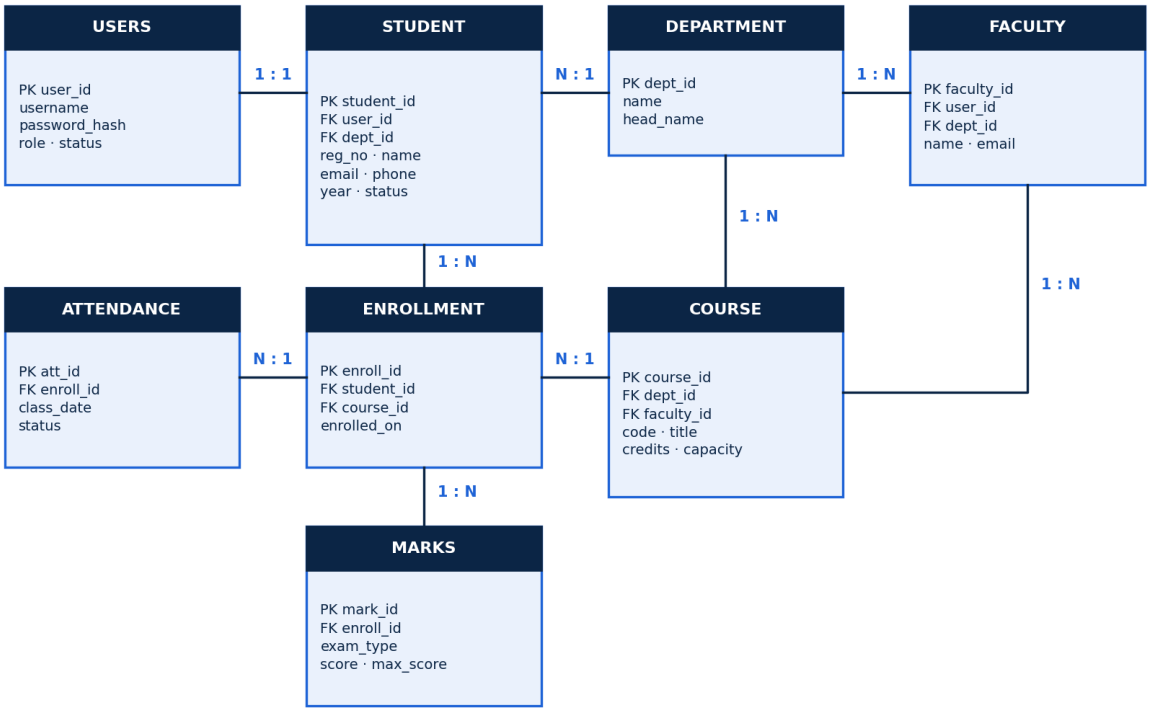


Figure 6.3: *Entity-relationship diagram*

6.5 Database Design

The two central tables are described below; the remaining tables follow the attributes shown in the entity-relationship diagram. Primary keys are indexed automatically, and foreign keys enforce referential integrity so that an enrolment can never refer to a missing student or course.

Table 6.1: Table STUDENT

Column	Type	Constraint / description
student_id	INT	Primary key, auto-increment
user_id	INT	Foreign key to USERS
dept_id	INT	Foreign key to DEPARTMENT
reg_no	VARCHAR(20)	Unique registration number
name	VARCHAR(120)	Not null, indexed for search
email · phone	VARCHAR	Contact details
year · status	INT · VARCHAR(15)	Year of study; ACTIVE, ALUMNI or BLOCKED

Table 6.2: Table MARKS

Column	Type	Constraint / description
mark_id	INT	Primary key, auto-increment
enroll_id	INT	Foreign key to ENROLLMENT
exam_type	VARCHAR(30)	Internal 1, Internal 2, Model or Semester
score	DECIMAL(5,2)	Marks obtained, between 0 and max_score
max_score	DECIMAL(5,2)	Maximum marks for the assessment

6.6 Process Flow: Attendance, Marks and Result

Attendance and result processing is the heart of the system. Marks are validated before they are saved; the system then computes the total and grade, and finally decides the result by checking both the attendance percentage and the pass mark.

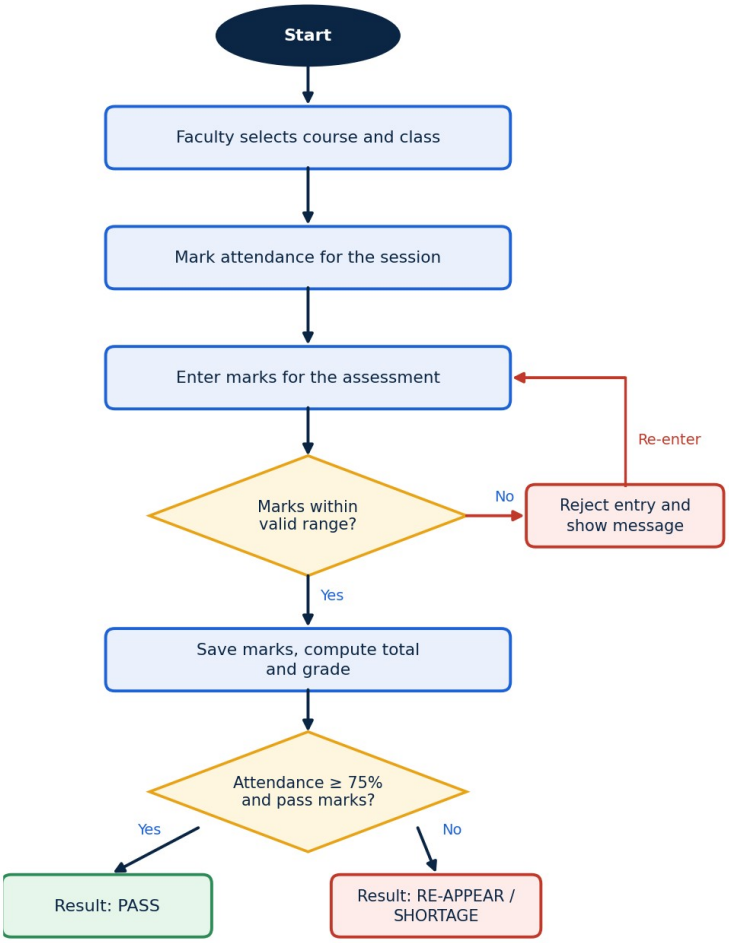


Figure 6.4: *Flowchart of attendance, marks entry and result processing*

CHAPTER 07

7. Implementation

7.1 Development Environment

The application was developed in Java using the Eclipse IDE. Spring Boot provided the application skeleton, embedded configuration and REST support; Hibernate with JPA handled persistence to a MySQL database; the web pages were built with HTML, CSS, JavaScript and JSP; Maven managed dependencies; and Git tracked every change to the source code. The application was packaged and deployed on Apache Tomcat.

7.2 Project Structure

The source code is organised by responsibility so that a developer can quickly find where a change belongs.

```
student-management-system/
  controller/   request handling (Student, Course, Attendance, Marks, Report)
  service/      business rules (enrolment, attendance, grading)
  repository/   Spring Data / Hibernate data access
  model/        JPA entities (Student, Course, Enrollment, Marks)
  security/     authentication and role configuration
  templates/    JSP / HTML views
  resources/    application properties, SQL scripts
```

7.3 Module Description

Table 7.1: Modules of the Student Management System

Module	Description
Authentication and access	Login and logout, password hashing and role-based authorisation for admin, faculty and student.
Student management	Add, edit, delete and search students; validate unique registration numbers; block or reactivate accounts.
Course and faculty management	Maintain departments, courses and faculty; assign courses to faculty.
Enrolment	Enrol students in courses with duplicate and capacity checks.
Attendance	Session-wise attendance entry and automatic percentage calculation.
Marks and results	Assessment marks entry, total, grade and pass or fail computation.
Reports	Attendance, result and enrolment reports with export option.

7.4 Key Implementation Logic

Three pieces of logic illustrate how business rules were implemented. The grade calculation and the attendance percentage are kept in the service layer so that they can be unit tested independently of the web layer.

```
public String calculateGrade(double percentage) {
    if (percentage >= 90) return "O";
    if (percentage >= 80) return "A+";
    if (percentage >= 70) return "A";
    if (percentage >= 60) return "B+";
    if (percentage >= 50) return "B";
    if (percentage >= 40) return "C";
    return "RA";
}

public double attendancePercentage(long present, long conducted) {
    if (conducted == 0) return 0.0;
    return (present * 100.0) / conducted;
}
```

The enrolment operation runs inside a single database transaction, so either the enrolment record is created and the seat count updated together, or neither happens.

```
@Transactional
public Enrollment enrollStudent(Long studentId, Long courseId) {
    Student s = studentRepo.findById(studentId).orElseThrow();
    Course c = courseRepo.findById(courseId).orElseThrow();
    if (enrollRepo.existsByStudentAndCourse(s, c))
        throw new AlreadyEnrolledException();
    if (c.getEnrolled() >= c.getCapacity())
        throw new CourseFullException();
    c.setEnrolled(c.getEnrolled() + 1);
    return enrollRepo.save(new Enrollment(s, c, LocalDate.now()));
}
```

> **NOTE**
> The snippets above are simplified illustrations of the approach used in the project. Class and method names in the final source code may differ slightly.

7.5 Security Implementation

- **Authentication:** users sign in with credentials verified against salted password hashes.
- **Authorisation:** Spring Security restricts every URL and action by role.
- **Input validation:** server-side validation prevents invalid marks, duplicate registration numbers and malformed e-mail addresses.
- **SQL injection protection:** all database access uses parameterised queries through JPA and Hibernate.
- **Session handling:** sessions are invalidated on logout and expire after a period of inactivity.

7.6 Version Control and Deployment

Source code was managed with Git, with regular commits describing each change so that the history of the project is easy to follow and any change can be reverted if required. For deployment, the application was packaged as a web archive and deployed on Apache Tomcat, with the MySQL database created from the SQL scripts included in the project.

CHAPTER 08

8. Testing

8.1 Objectives of Testing

Testing was carried out to confirm that the system behaves as specified, that business rules are enforced under normal and unusual inputs, and that unauthorised users cannot reach restricted functions. Test cases were designed from the functional requirements in Chapter 5 so that every requirement is covered by at least one scenario.

8.2 Testing Strategy

Table 8.1: Levels of testing

Level	Focus	Tools / method
Unit testing	Individual methods such as grade calculation and attendance percentage	JUnit
Integration testing	Interaction between controller, service and database layers	JUnit, Spring Boot test support
System testing	Complete workflows such as enrolment, attendance and result generation	Manual test scripts
Security testing	Role restrictions, session expiry and invalid input handling	Manual verification
User acceptance testing	Usability and correctness from the point of view of the mentor and users	Demonstration and feedback

8.3 Sample Test Cases

Table 8.2: Representative test cases and expected results

ID	Scenario	Input / condition	Expected result
TC-01	Valid login	Correct username and password	User reaches the dashboard for the role
TC-02	Invalid login	Wrong password	Error message; access denied
TC-03	Add student	All mandatory fields filled	Student saved and visible in the list
TC-04	Duplicate reg. no.	Registration number already exists	Validation error; record not saved
TC-05	Enrol student	Not enrolled, seats available	Enrolment created; seat count updated
TC-06	Duplicate enrolment	Student already in the course	Enrolment rejected with a clear message
TC-07	Course full	Enrolled count equals capacity	Enrolment rejected
TC-08	Invalid marks	Score above maximum or negative	Validation error; marks not saved
TC-09	Grade computation	Score of 82 out of 100	Grade A+ assigned

ID	Scenario	Input / condition	Expected result
TC-10	Role restriction	Student opens an admin URL	Access denied

8.4 Defect Handling

Defects found during testing were recorded with the steps needed to reproduce them, fixed in the relevant layer, and re-tested together with the surrounding functionality to make sure that the fix had not affected anything else. Typical defects of this kind are boundary errors in grade bands, missing validation messages and menu options displayed to the wrong role. Using JUnit for the business rules allowed such defects to be caught quickly and kept from returning.

CHAPTER 09

9. Application Walkthrough and Results

This chapter walks through the main screens of the Student Management System. The screens follow the layout described in Chapter 6: a navigation menu on the left that shows only the options permitted for the logged-in role, a header with the current user, and a content area for the active function.

9.1 Login Page

Every user begins at the login page, which asks for a username and password verified against salted password hashes. After successful authentication the user is taken to a dashboard that matches the role, and unauthorised users are denied access to every other page.

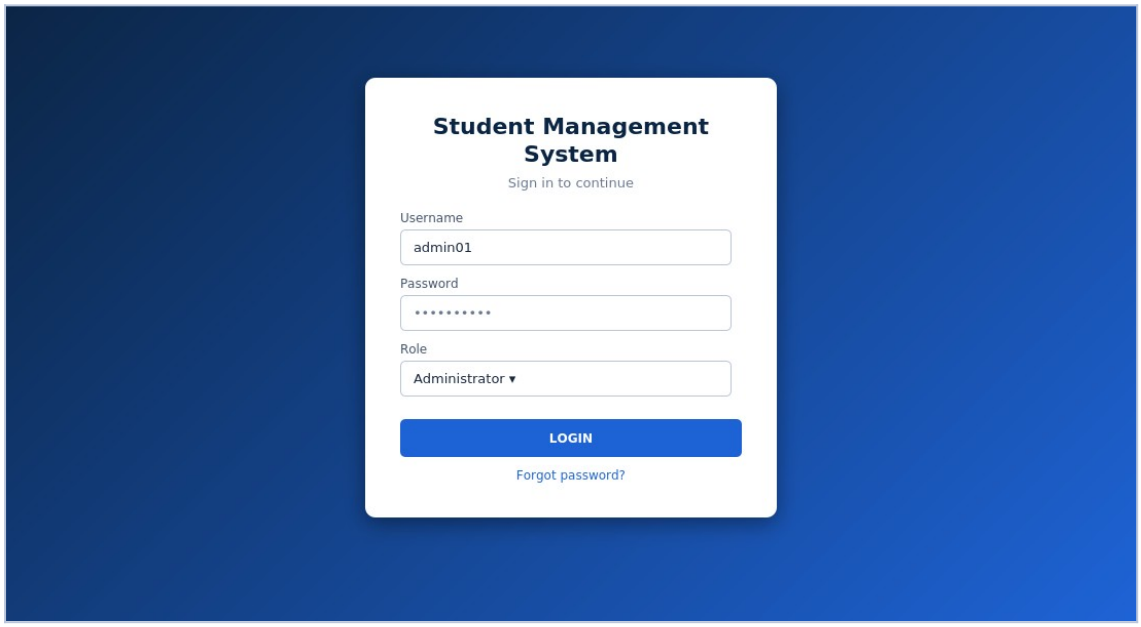


Figure 9.1: *Login page*

9.2 Dashboard

The dashboard gives the administrator a quick picture of the institution: summary tiles for students, faculty, courses and enrolments, with shortcuts to the most common tasks.

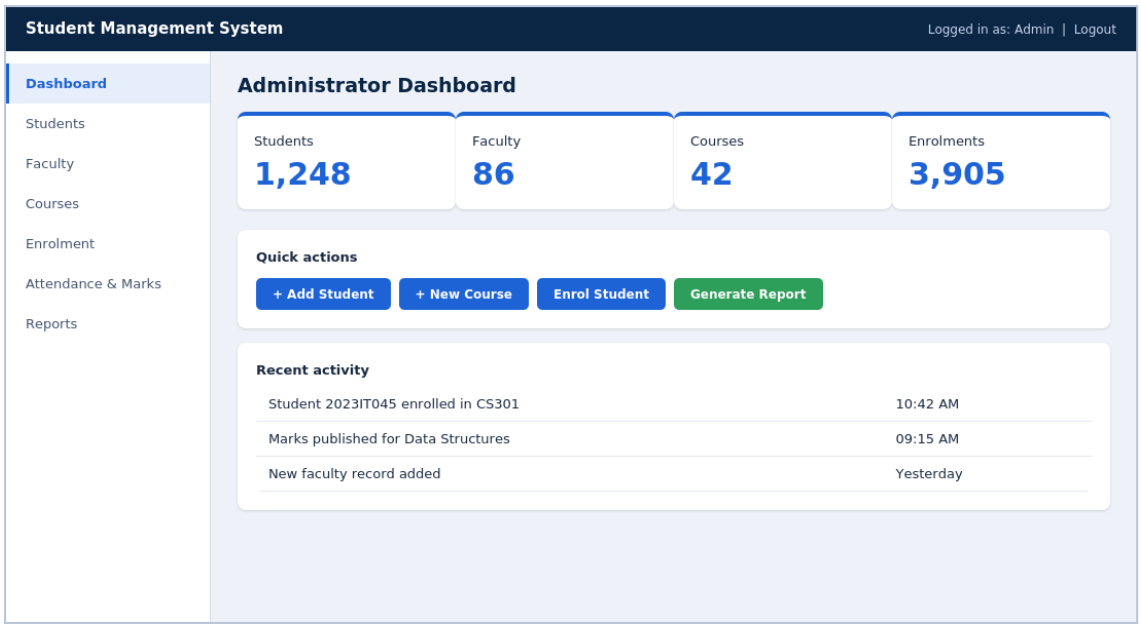


Figure 9.2: *Dashboard with summary tiles and quick actions*

9.3 Student Management

The student list shows each student with registration number, name, department, year and status. The search box and filters narrow the list, and authorised users can add, edit or delete a record while the registration number is validated to prevent duplicates.

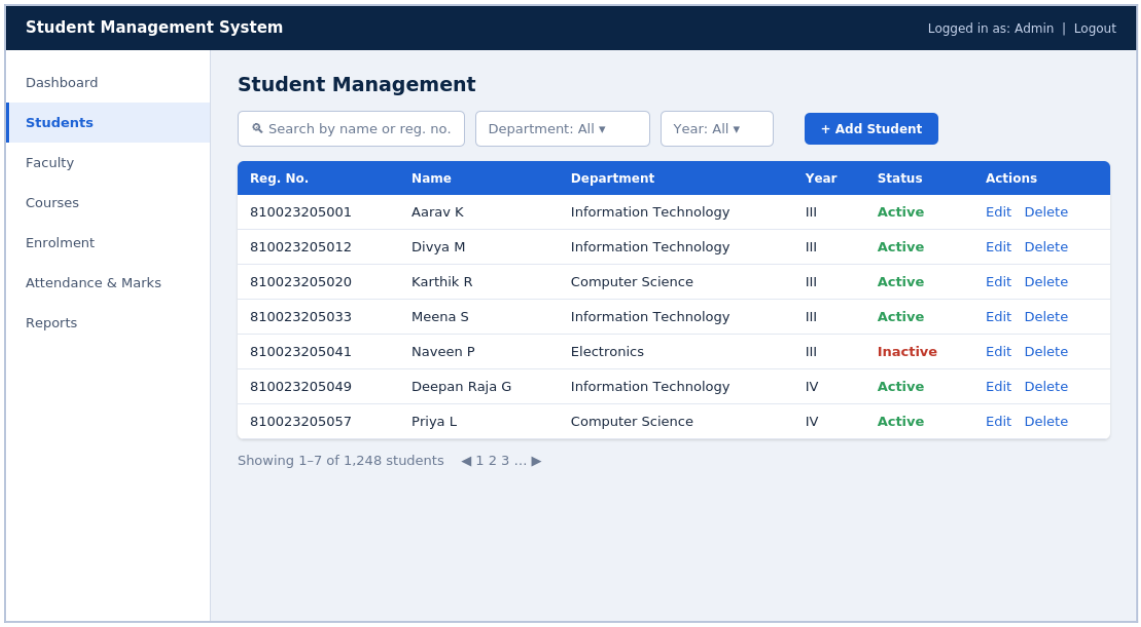


Figure 9.3: *Student list with search, filters and status*

9.4 Course Enrolment

On the enrolment screen the administrator selects a student and a course. The system shows the seats available and rejects duplicate or over-capacity enrolments with a clear reason.

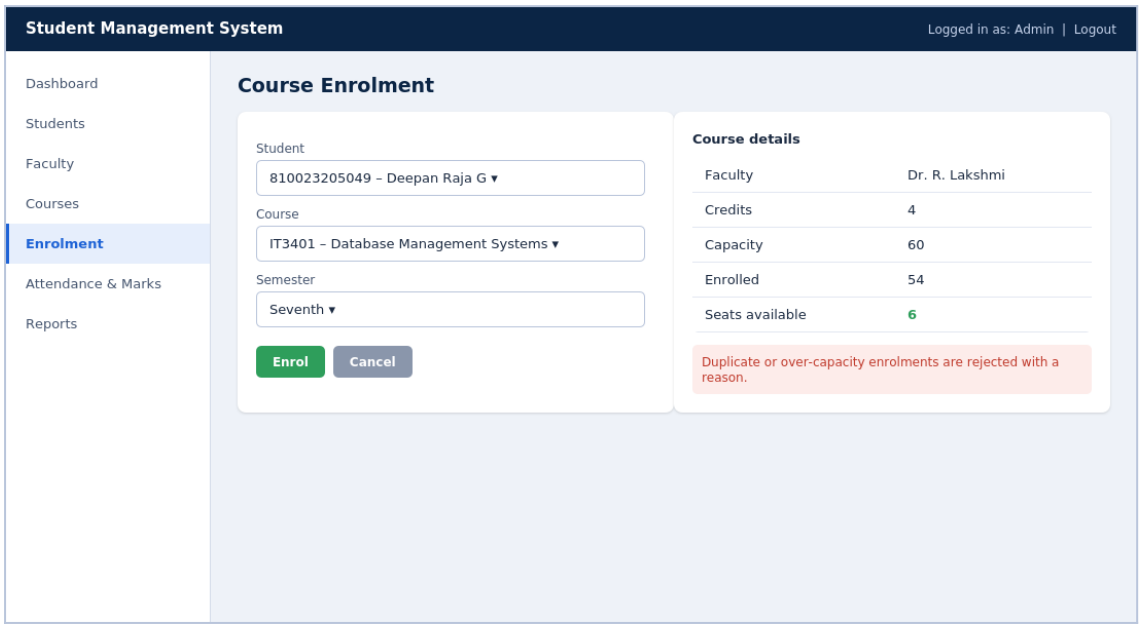


Figure 9.4: *Course enrolment screen*

9.5 Attendance and Marks Entry

Faculty select a course and session, mark each student present or absent, and enter assessment marks. Percentages, totals and grades are calculated automatically and displayed before the result is published.

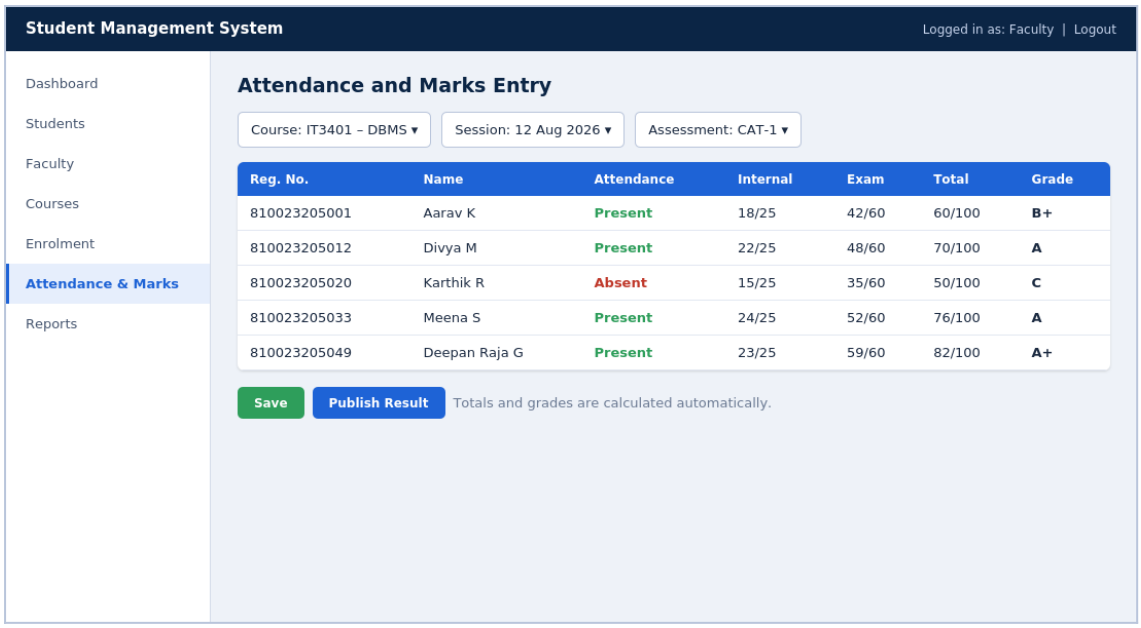


Figure 9.5: *Attendance and marks entry screen*

9.6 Reports and Student View

The reports module summarises attendance, results and enrolment for a selected period or course. Students see their own profile, attendance percentage and results through a read-only view.

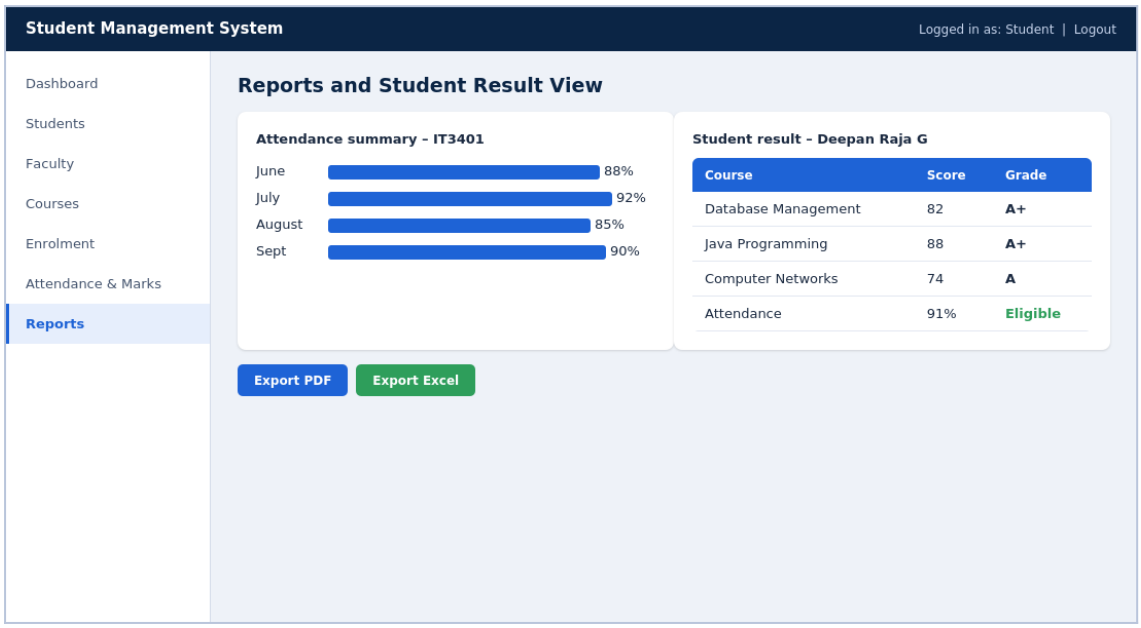


Figure 9.6: *Reports screen and student result view*

9.7 Requirement Coverage

The table below maps the functional requirements of Chapter 5 to the screens in which they are realised, showing that every requirement is covered by the application.

Table 9.1: Coverage of functional requirements by application screens

Requirement	Realised in	Screen
FR-01 Login and logout	Authentication module	Login page (Figure 9.1)
FR-02 Student management	Student management module	Student list (Figure 9.3)
FR-03 Course management	Course and faculty module	Dashboard shortcuts (Figure 9.2)
FR-04 Faculty management	Course and faculty module	Dashboard shortcuts (Figure 9.2)
FR-05 Enrolment	Enrolment module	Course enrolment (Figure 9.4)
FR-06 Attendance	Attendance module	Attendance entry (Figure 9.5)
FR-07 Marks entry	Marks and results module	Marks entry (Figure 9.5)
FR-08 Grade and result	Marks and results module	Marks entry (Figure 9.5)
FR-09 Student view	Student view	Result view (Figure 9.6)
FR-10 Reports	Reports module	Reports (Figure 9.6)

9.8 Results Achieved

Table 9.2: Project objectives and outcomes

Objective	Outcome
Searchable records	Students, faculty and courses can be added, updated and found by multiple criteria

Objective	Outcome
Automated enrolment	Duplicate and over-capacity enrolments are prevented automatically
Attendance tracking	Session-wise attendance with automatic percentage and shortage detection
Result automation	Totals, grades and pass or fail status are computed from the configured rules
Secure access	Role-based access protects every function
Reporting	Attendance, result and enrolment reports are available

9.9 Usability Observations

- **Consistent navigation:** the same menu, header and colour scheme are used on every page.
- **Colour-coded status:** green, amber and red badges make attendance shortage and results recognisable at a glance.
- **Immediate feedback:** validation messages appear as soon as a field is incorrect, so mistakes are caught before saving.
- **Less manual work:** percentages, totals and grades are calculated automatically, reducing data entry and errors.
- **Role-aware menus:** students and faculty see only the functions they are permitted to use.

CHAPTER 10

10. Learning Outcomes and Challenges

10.1 Technical Skills

- Designing object-oriented solutions using classes, interfaces, inheritance and polymorphism.
- Using collections, generics, file handling and multithreading appropriately.
- Connecting applications to relational databases using JDBC and Hibernate and writing efficient SQL.
- Building web applications with Servlets, JSP and Spring Boot, and exposing REST services.
- Applying authentication and authorisation with Spring Security.
- Writing unit and integration tests with JUnit and debugging with IDE tools.
- Using Git for version control and Apache Tomcat for deployment.

10.2 Professional Growth

Beyond technology, the internship built communication, teamwork, time management, problem solving, documentation and professional discipline. Regular discussions with the industry guide, code reviews with colleagues and module-wise planning of the project all contributed to this growth.

10.3 Reflection

Context. The internship was a valuable month that gave direct exposure to a professional working environment and showed how an organisation expects its members to conduct themselves.

My role. As an intern, my responsibility was to follow the organisation's rules, to learn from the team as a newcomer, and to contribute to the Student Management System within the scope assigned.

Action taken. I applied the knowledge and skills from my specialisation to the tasks given, studied new technologies where the project demanded them, and sought guidance whenever a problem remained unresolved.

Performance. Colleagues and mentors supported me through every challenge and were available whenever I needed assistance, which helped me progress steadily.

Gain. I understood the importance of orientation in settling into a new environment and improved my understanding of individual and group behaviour in an organisation. Technically, I gained confidence in building a complete application end to end.

10.4 Challenges and Solutions

Table 10.1: Challenges faced and solutions adopted

Challenge	Solution adopted
Learning many technologies in a short time	Followed the structured plan, practised each concept immediately in small programs and revised from the official documentation.
Mapping Java objects to relational tables	Studied Hibernate and JPA annotations and verified mappings against the database design using small test programs.

Challenge	Solution adopted
Keeping enrolment and seat counts consistent	Wrapped enrolment operations in database transactions.
Correct grade and attendance calculation	Isolated the logic in service methods and covered the grade boundaries with unit tests.
Restricting features by user role	Configured Spring Security rules by URL pattern and hid menu options not permitted for a role.
Integrating front end and back end	Agreed on clear request and response structures and tested integration module by module.
Managing code changes	Used Git with small, frequent commits and discussed changes in code reviews.

CHAPTER 11

11. Conclusion and Future Scope

11.1 Conclusion

The internship at Fantasy Solution was a rewarding and well-structured learning experience. In thirty days it covered the journey from the fundamentals of Java to the development of a complete, database-backed web application. Through daily practice, coding exercises and project work, I gained solid skills in object-oriented programming, web development, database connectivity, security, testing and deployment.

The Student Management System developed during the internship demonstrates how these skills combine to solve a practical problem. It replaces manual registers with a secure and reliable system that manages students, courses, enrolment, attendance, results and reports through a role-based interface. Equally important, the internship taught professional habits: working to a plan, communicating clearly, receiving feedback and documenting work.

I am grateful for the opportunity, and I believe that the experience has prepared me well for the challenges of software development in my future career.

11.2 Future Enhancements

- **Notifications:** automatic e-mail or SMS alerts for attendance shortage and published results.
- **Fee management:** fee records, receipts and online payment through a secure gateway.
- **Timetable management:** automatic timetable generation and conflict detection.
- **Parent portal:** read-only access for parents to track attendance and performance.
- **Analytics dashboard:** charts for performance trends and course-wise results.
- **Online assessments:** quizzes and assignments with automatic evaluation.
- **Multi-campus support:** a shared database across several campuses.
- **Mobile application:** a companion Android app for students and faculty.

References

6. Oracle Corporation. Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/
7. VMware / Broadcom. Spring Boot and Spring Framework Reference Documentation. https://spring.io/projects/spring-boot
8. Red Hat. Hibernate ORM User Guide. https://hibernate.org/orm/documentation/
9. Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/
10. Apache Software Foundation. Apache Tomcat Documentation. https://tomcat.apache.org/
11. JUnit Team. JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
12. Chacon, S. and Straub, B. Pro Git. https://git-scm.com/book/
13. Elmasri, R. and Navathe, S. Fundamentals of Database Systems. Pearson.
14. Fantasy Solution. Company website. www.fantasysolution.in